SYSTEM DESIGN · TAKE-HOME ASSIGNMENT

Exactly-Once Publisher

200,000 SEO articles a day to customer sites:
Shopify, WordPress, Webflow, Wix

THE TASK

What we need to build

200,000

articles a day to customer sites

2.3 per second on average and about 25 at peak, when everyone schedules for 9:00

Any API call can fail

Timeout, 5xx, dropped connection. And you can’t tell whether the call went through.

The service can be restarted

At any moment, in the middle of publishing.

Every article goes live exactly once

After a restart: no duplicates and nothing missing.

DATA

An article: text and images

Problem. An article has text (10–30 KB) and images (up to 3 MB). Images bloat the database and slow down backups; text in a separate store costs an extra request per article.

Solution. Text lives in Postgres: the worker gets it with the same query that takes the job. Images go to S3, and the database keeps only a link to the file.



200,000 articles × 20 KB ≈ 4 GB of text a day. That’s fine for Postgres: it compresses large fields and stores them outside the row.

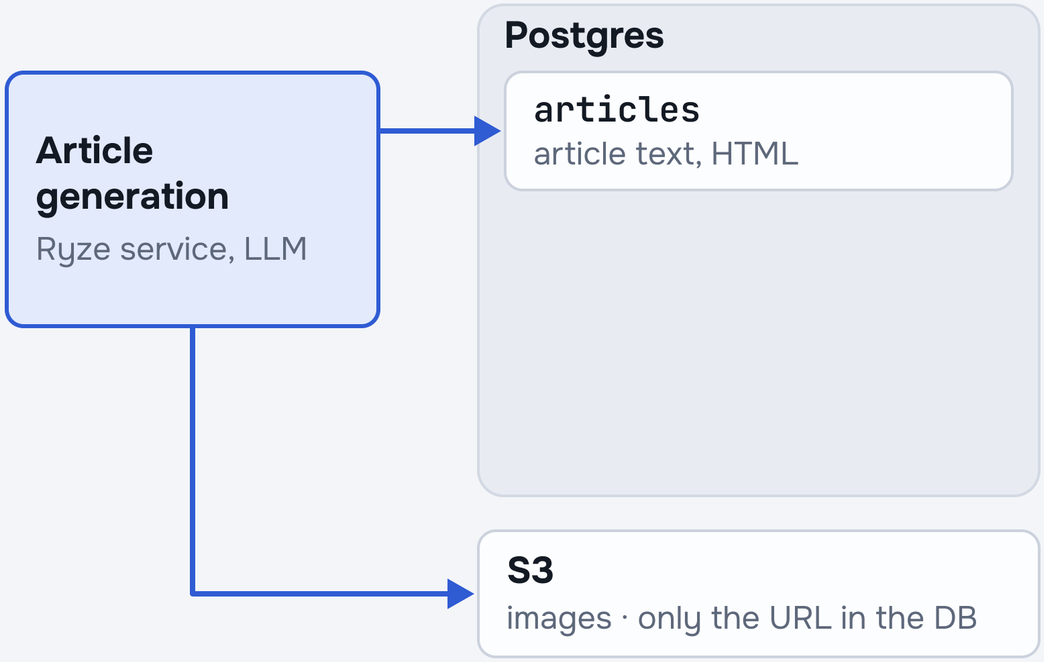
Images are binary and heavy. They belong in object storage and are easy to serve through a CDN.

DATA

Where articles come from

Problem. A separate generation service writes the articles. We need to agree on when an article is “ready” and where our work begins.

Solution. Generation puts the text in Postgres and the images in S3, and gives the article a permanent uid. Once the article is written, publishing it is our job.



Generation is an existing service. We don’t change it, we only agree on the format: every article has a stable **uid**.

That uid later becomes the marker on the published post. It’s how we recognise our post on the customer’s site.

REQUIREMENTS

What the system must do

A Exactly once

Every article goes live once: no duplicates, nothing missing.

B Failures lose no work

Timeout, 5xx, restart, crashed worker: we resume where we stopped.

C Every site’s rate limit

We stay within platform limits, and a big customer doesn’t slow down the rest.

D New platforms, no rewrites

Shopify, WordPress, Webflow, Wix and the next one: the core doesn’t know about them.

200,000 a day, peak ~25/s

live within ~15 minutes

customers see each article’s status

THE HARD PART

We sent “create post” and the response never came.

Was the post created? There is no way to know. Retry, and we may get a duplicate. Don’t, and the article may never go live. None of these platforms takes an idempotency key.

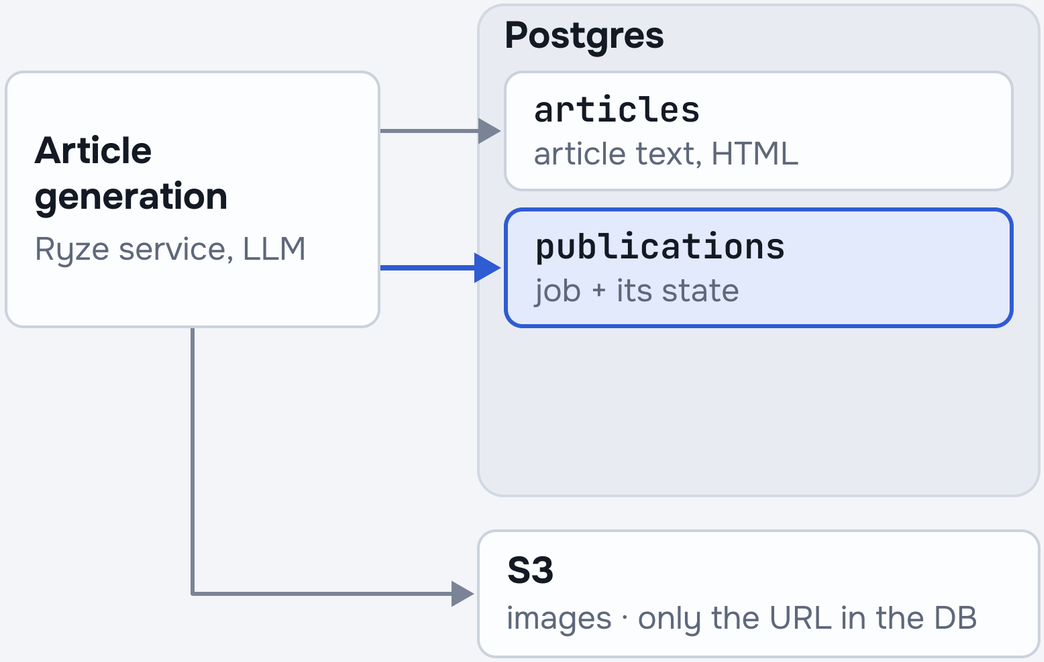
So the goal is different: not “call the API once” but “readers see the article once”.

STEP 1 · QUEUE

A job queue in the same database

Problem. We need to remember which articles haven't gone live on which sites. The list must survive a restart, and re-running the daily batch must not create duplicates.

Solution. A publications table: one row is one article for one site, together with its status. The (article, site) pair is unique, so inserting it again adds nothing.



Generation enqueues with **INSERT … ON CONFLICT DO NOTHING**: "insert the row, or silently skip it if the pair is already there".

Why not Kafka: you can't update a message in a queue ("step 2 done, post 42"). Progress would still have to live in the database, leaving two systems to keep consistent.

STEP 2 · WORKERS

Workers pick up jobs

Problem. Someone has to take jobs and call the sites' APIs. If it crashes mid-way, the job must not stay stuck "in progress" forever.

Solution. When a worker takes a job, it immediately sets a retry time: "in 30 s". Done: it writes "done". Dead: the time comes and another worker picks the job up.



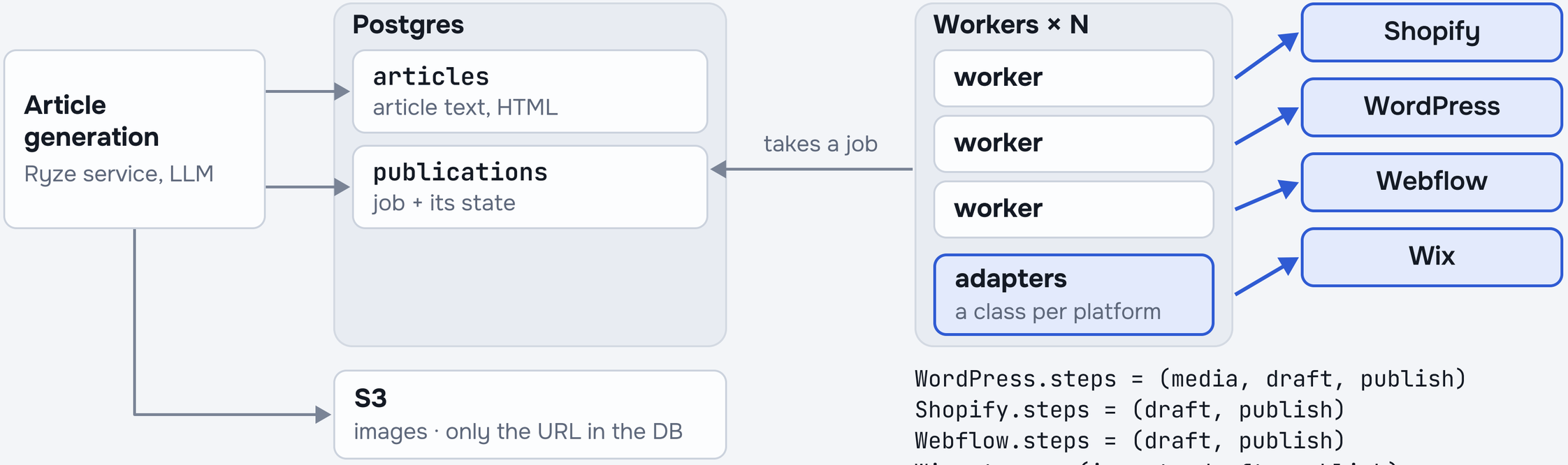
A worker is a program in a container, and several run at once. It keeps nothing itself: all progress is in the DB, so any worker can be killed and another carries on.

STEP 3 · PLATFORMS

One adapter per platform

Problem. Each platform has its own API: its own requests, order of actions and errors. Put that in the shared worker code and every new platform will break it.

Solution. Platform-specific code lives in an adapter: it declares its steps and runs them. The worker just follows those steps and knows no platform.




```
WordPress.steps = (media, draft, publish)
Shopify.steps = (draft, publish)
Webflow.steps = (draft, publish)
Wix.steps = (import, draft, publish)
```

STEP 4 · PROGRESS

Remember where we stopped

Problem. Publishing is several requests in a row. If a worker crashes after the second one and we saved nothing, the next one starts over: uploads the image again and creates a second post.

Solution. After every step we write to the article's row what is done and what the site returned. A new worker reads the row and resumes at the same step.

```
WordPress.steps = (media, draft, publish)
```

step 1 · media

Upload the image

```
to DB: step = draft,
media_id = 17
```

→

step 2 · draft

Create a draft

```
to DB: step = publish,
post_id = 42
```

→

step 3 · publish

Publish post 42

```
to DB: status = done
```

The worker crashed after step 2

The row already says step = publish and post_id = 42. The new worker goes straight to step 3: no second image upload, no second post.

Two workers took the same article

The write carries a condition, "only if the step is still draft" (compare-and-set). The first worker's write lands; for the second the condition no longer holds, and it walks away.

```
UPDATE … WHERE step = 'draft'
```

STEP 5 · THE WHOLE LOOP

The worker pipeline

What the worker does	What it writes to the DB
1. Leases a customer site	site: lease_until = +60 s
2. Takes an article of that site whose run_at has come	attempts + 1, run_at = + backoff
3. media: uploads the image to the site	step = draft, media_id
4. draft: looks for its post by marker; if none, creates a draft without a slug	step = publish, post_id
5. publish: trashes leftover drafts, publishes the post with its slug	status = done, url
6. Takes the next article; releases the site after 20 s	site: lease_until = now

While the steps run, the article is in **status = pending**. Every step write carries the condition **WHERE step = current** (CAS).

When something goes wrong

Timeout, 5xx
last_error; retried automatically when run_at comes

429 “too many requests”
attempts - 1; site paused until Retry-After

401 / 403
attempts - 1; site: status = needs_reauth

4xx or out of attempts
status = failed

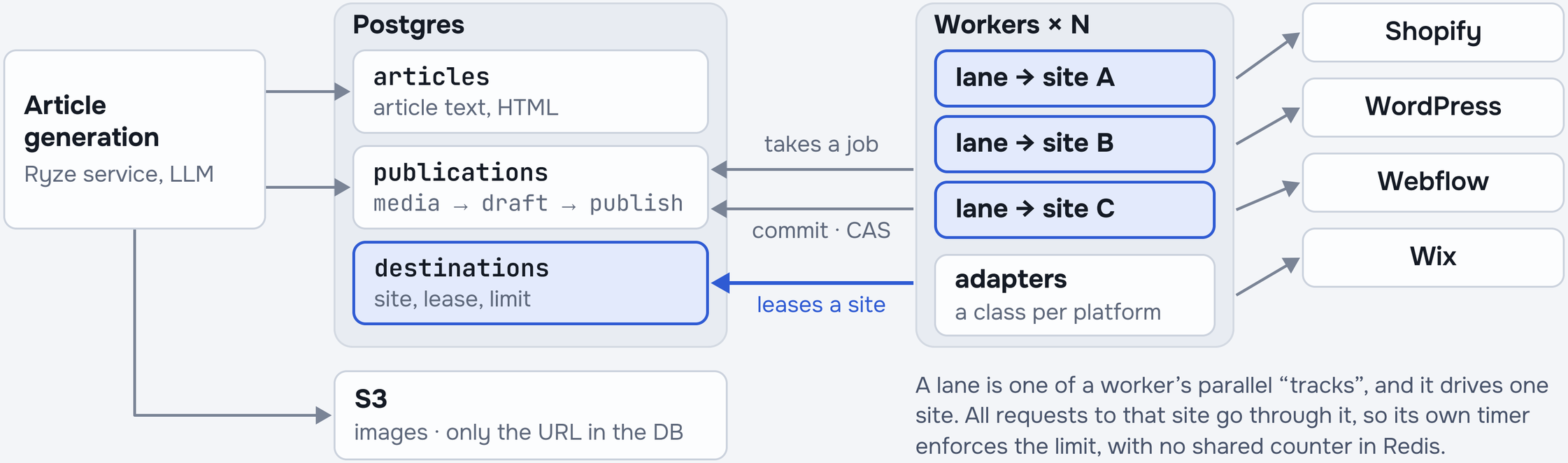
CAS updated 0 rows
another worker already wrote the step; this one leaves

STEP 6 · RATE LIMITS

Lease the site, not the article

Problem. Every site has a request limit; Shopify allows about 2 per second. If 20 workers hit one site at once, it answers “too many requests” and other customers wait.

Solution. A worker “leases” the site: it writes “this site is mine until 12:00:30” to the DB and works with it alone, pausing between requests. After 20 s it hands the site on.

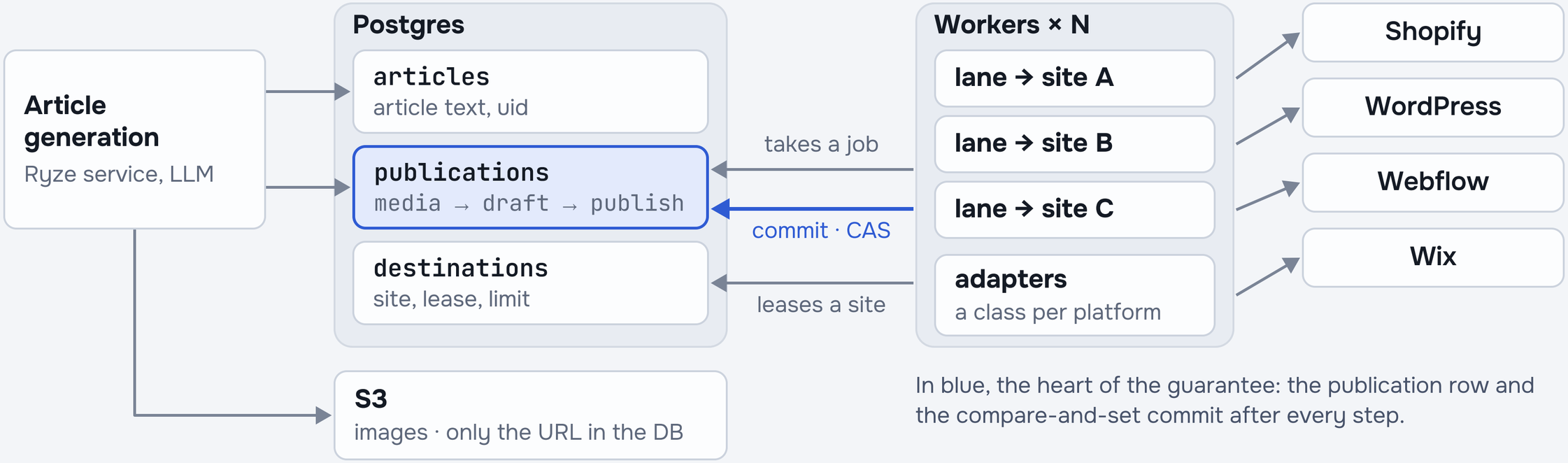


A lane is one of a worker’s parallel “tracks”, and it drives one site. All requests to that site go through it, so its own timer enforces the limit, with no shared counter in Redis.

SUMMARY

The whole system

Three tables, one worker loop and an adapter per platform. No Kafka, Redis or Temporal.



In blue, the heart of the guarantee: the publication row and the compare-and-set commit after every step.

REPOSITORY

What’s in the code

github.com/MaksimovDania/exactly-once-publisher

publisher.py
The worker, ~150 lines: site lease, taking a job, adapter steps, CAS commits, retries and pauses.

platforms.py
The adapter contract, error mapping into 4 kinds, site pacing and the WordPress adapter.

schema.sql
Three tables: articles, publications, destinations.

fake_platform.py
A fake WordPress that breaks on purpose: errors, hangs, 429s, slow search.

chaos.py
End-to-end test: runs the workers under chaos and checks every article went live exactly once.

README.md · REVIEW.md
The technical design with diagrams, and an independent review with the bugs it found, now fixed.

```
docker compose up -d && python chaos.py
```

TESTING

How I tested it

Chaos test

20 sites, 500 articles, the queue filled twice. Every 1–3 s a worker is killed with kill -9 or frozen past its lease.

A site that breaks

Fake WordPress: 10% errors before and after the write, hangs, 429s, lagging search, WordPress slug rules.

Two workers race

Both start from the same snapshot and each creates a draft. Exactly one is published; the other walks away.

Restore from backup

The article’s row is rolled back, then lost and enqueued again. The post is found by uid, no duplicate.

Deliberate breakage

Removed CAS, put the slug back in the draft, plus 3 more changes. The test fails on every one of the five.

Independent review

13 harsh runs, up to 50% failures and database disconnects. Fixes re-checked on a real WordPress 7.1.3.

Every run ends with one check: each article is live exactly once, at its own address.

RESULTS

What we did and what we got

1. **Generated 500 articles** for 20 sites. Every tenth has a Cyrillic address.
2. **Enqueued them twice**, as if the daily batch ran again.
3. **Ran a fake WordPress that breaks:** 10% of requests fail before the write, after it, or hang past the timeout.
4. **Set rate limits:** a site takes 10 requests per second; some sites we configured for 20, so they get 429s.
5. **Started 4 workers with 8 lanes each** and every 1–3 s killed one (kill -9) or froze it past its lease. One site's password was revoked and restored 15 s later.
6. **Checked every article on the site:** whether it went live, and how many times.

SUMMARY

500 / 500

articles went live exactly once, at their own address: 0 duplicates, 0 missing

25 + 7 workers killed and frozen, no work lost

163 429 responses: the site paused, no attempts burned

0 attempts used by the site with the revoked password while it waited

The test also forces two workers to race for one article and replays a restore from backup. It caught all 5 deliberate code breakages.